

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2019/2020

SEPTEMBER EXAMINATION

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVING

THURSDAY, 12 SEPTEMBER 2019

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONS)
BACHELOR OF INFORMATION SYSTEMS (HONS)
INFORMATION SYSTEMS ENGINEERING
BACHELOR OF INFORMATION TECHNOLOGY (HONS)
COMMUNICATIONS AND NETWORKING
BACHELOR OF INFORMATION TECHNOLOGY (HONS)
COMPUTER ENGINEERING

This question paper consists of **THREE (3)** questions in Section A and **TWO (2)** questions in Section B.

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**. Each question carries 25 marks.

Should a candidate answer more than ONE (1) question in Section B, marks will only be awarded for the **FIRST** questions in the order the candidate submits the answers.

Candidates are allowed to use a non-scientific calculator.

Answer questions only in the answer booklet provided.

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVINGSECTION A

- Q1. (a) Based on the definition in the program below, what is the output of the following cout statements? If the cout statement causes error, then provide the reason for the error.

```

struct person{
    char name[30];
    int age;
    char phone[10];
    char gender;
    person(char *, int, char *, char); //name, age, phone, gender
};

void main(){
    person p("Alice Wong", 23, "012-3789022", 'F');
    person *ptr;
    ptr = &p;
}

```

- (i) cout << "\n" << ptr->name[3]; (2 marks)
- (ii) cout << "\n" << ptr->gender[0]; (2 marks)
- (iii) cout << "\n" << p.phone[6]; (2 marks)
- (iv) cout << "\n" << *(ptr.age); (2 marks)
- (b) A linked list with 4 nodes are given in Figure 1 below. Redraw the linked list after the execution of following statements. [Definitions for the functions used below can be found in Appendix A.]



Figure 1: Linked List

```

list.insert(3, 10);
list.set(4, 12);
list.remove(5);
list.set(2, 8);
list.insert(2, 7);

```

(10 marks)

- (c) Based on the definition for linked list in Appendix A, write a function **bool List::swapValues(int n)** to swap value in node **n** with the first node in the list. **n** must be a valid value since it indicates the node position in the list. Use function **find()** to find node in position **n**. Function will return **false** if **n** is not valid or the function is unable to find node **n**. Example output if **n = 5** is given below:

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVING

Q1. (c) (Continued)

Example:**List Content Before Swap Value: {3, 7, 10, 12, 16, 18}****List Content After Swap Value: {16, 7, 10, 12, 3, 18}**

(7 marks)

[Total : 25 marks]

- Q2. (a) What is the output of the following program? [Definition for the class Stack can be found in Appendix B.]

```

Stack stack1, stack2;
char *a[5] = { "Pam", "John", "Tom", "Ken", "Yvonne" };
char *str1, *str2;

for (int x = 0; x < 5; x++) {
    if (a[x][0] > 'K') {
        cout << "\n" << a[x] << " push to stack1";
        stack1.push(a[x]);
    }
    else {
        cout << "\n" << a[x] << " push to stack2";
        stack2.push(a[x]);
    }
}

cout << "\n\nStack 1: \n";
while (!stack1.empty()) {
    if (stack1.pop(str1))
        cout << str1 << "\t";
}

cout << "\n\nStack 2: \n";
while (!stack2.empty()) {
    if (stack2.pop(str2))
        cout << str2 << "\t";
}

```

(10 marks)

- (b) What is the output of the following program segment? [The Queue definition is in Appendix C.]

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVINGQ2. (b) (Continued)

```

void funcA(char c, Queue *q1, Queue *q2) {
    char *temp = new char[30];
    int cnt = q1->size();

    for (int i = 1; i <= cnt; i++) {
        q1->dequeue(temp);

        if (temp[0] == c)
            q2->enqueue(temp);
        else
            q1->enqueue(temp);
    }
}

int main() {

    Queue queue1;
    Queue queue2;
    char *name[8] = { "Apple", "Orange", "Guava", "Banana", "Eggfruit",
                     "Avocado", "Olive", "Papaya" };
    char key[5] = { 'A', 'E', 'T', 'O', 'U' };
    char *temp = new char[30];

    for (int i = 0; i < 6; i++)
        queue1.enqueue(name[i]);

    for(int i = 0; i < 5; i++)
        funcA(key[i], &queue1, &queue2);

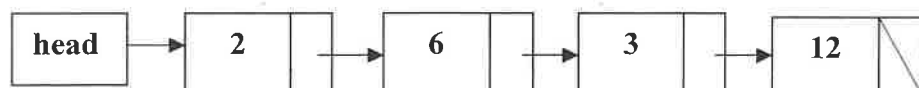
    cout << "\nQueue 1 Contains: ";
    while (!queue1.empty()) {
        queue1.dequeue(temp);
        cout << temp << " ";
    }

    cout << "\nQueue 2 Contains: ";
    while (!queue2.empty()) {
        queue2.dequeue(temp);
        cout << temp << " ";
    }
}

```

(6 marks)

- (c) The following questions are based on the list shown in Figure 2 below.

**Figure 2: List With Four Items**

This question paper consists of 5 questions on 12 printed pages.

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVING**Q2. (c) (Continued)**

- (i) Assume that the list in Figure 2 as a stack. Redraw the stack after the following statements are executed. (4 marks)

```

stack.pop(item) //item is of type int
stack.pop(item)
stack.push(9);
stack.pop(item);
if(item > 10)
    stack.push(10);
stack.push(13%2);
stack.push(item/2);

```

- (ii) Assume that the list in Figure 2 as a queue where the head pointer is pointing to first node with value 2 and tail pointer is pointing to last node with value 12. Redraw the queue after the following statements are executed. Show the head and tail pointers in your queue. (5 marks)

[Total : 25 marks]

```

//both item1 and item2 are type int
queue.dequeue(item1);
queue.enqueue(7);
queue.enqueue(15);
queue.dequeue(item2);
if(item1%2 == 1)
    queue.enqueue(item1);
else
    queue.enqueue(item2);

```

- Q3. (a) (i) The following numbers will be sorted using Quick Sort in ascending order. The first partition of the list is shown in Figure 3 below. The pivot value is in the box that is shaded. Redraw Figure 3 and fill in the value for the boxes with X. Provide the index value in each step also. [The partition function is given in Appendix D.] (10 marks)

76 13 90 65 6 37 82

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVING

Q3. (a) (i) (Continued)

76	13	90	65	6	37	82	index = 0
65	X	X	X	X	X	X	index = ?
65	X	X	X	X	X	X	index = ?
65	X	X	X	X	X	X	index = ?
65	X	X	X	X	X	X	index = ?
65	X	X	X	X	X	X	index = ?
65	X	X	X	X	X	X	index = ?
65	X	X	X	X	X	X	index = ?
X	X	X	65	X	X	X	index = ?

Figure 3: First Partition in a Quick Sort

- (ii) What are the sorting methods for A, B, C, D, and E that have the best case and worst case scenario shown in the table below? If there is no sorting method matches with the best and worst case scenario, just put "Not Available".

Bubble Sort Insertion Sort Merge Sort Quick Sort

Sorting Methods	Best Case	Worst Case
A	$O(N)$	$O(N^2)$
B	$O(N \log_2 N)$	$O(N \log_2 N)$
C	$O(N^2)$	$O(N^2)$
D	$O(\log_2 N)$	$O(N)$
E	$O(N \log_2 N)$	$O(N^2)$

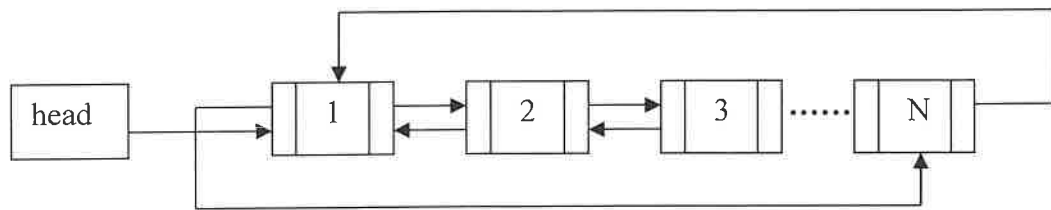
(5 marks)

- (b) Use Binary Search to search for the character 't' in the array below. Show the index values for start, mid and end for every step.

'a'	'c'	'e'	'h'	'k'	'o'	'r'	't'
-----	-----	-----	-----	-----	-----	-----	-----

(4 marks)

- (c) Figure 4 shows a sorted (in ascending order) circular doubly linked list. The list can store up to N nodes.

Q3. (c) (Continued)**Figure 4: Sorted Circular Doubly Linked List**

Provide the complexity in Big-O notation for the worst case scenario if the following operations are applied on the list in Figure 4 above. Justify your answer.

- (i) Finding the largest item in the list. (2 marks)
 - (ii) Calculate the total of all nodes value in the list. (2 marks)
 - (iii) Delete an item in the list according to value store in node. (2 marks)
- [Total : 25 marks]

SECTION B

- Q4. (a) (i) Draw the Binary Search Tree if the following characters are inserted into the tree from left to right one at a time. The tree is empty before the first character is inserted. (7 marks)

H L E J M B F A C G I D K

- (ii) The tree drawn in Q4(a)(i) is not balance if the node G is deleted. Specify which node is the deepest unbalance node and what case (LL, LR, ...) it belongs to. Draw the resultant tree after balancing. (6 marks)
 - (iii) Redraw the tree in Q4(a)(ii) (after balancing) if the node C is deleted. (5 marks)
- (b) The functions **treeFunction** and **treeFunction2** are given in Appendix E. Answer the following questions based on these two functions. [Definitions for **BTNode** and **BST** are given in Appendix E.]
- (i) What is the purpose of these two functions? (2 marks)
 - (ii) What will be printed if the tree drawn in Q4(a)(i) is used to call the function **treeFunction**? (5 marks)
- [Total : 25 marks]

Q5. (a) Figure 5.0 shows an undirected weighted graph A.

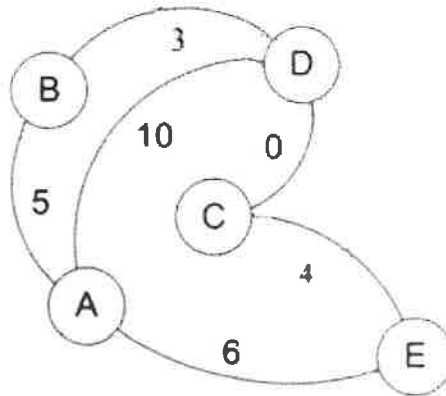


Figure 5.0 Undirected Weighted Graph A

- (i) Give the adjacency matrix and adjacency list representation for the graph in Figure 5. (6 marks)
 - (ii) Use **Breadth-First Search (BFS)** to traverse the graph in Figure 5.0 starting from vertex A. List out the order of the nodes visited. Visit the neighboring nodes according to alphabetical order. (3 marks)
 - (iii) Use **Depth-First Search (DFS)** to traverse the graph in Figure 5.0 starting from vertex A. List out the order of the nodes visited. Visit the neighboring nodes according to alphabetical order. (3 marks)
 - (iv) Find the **Shortest Path** from node A using **Dijkstra's algorithm**. Show the final graph after all the shortest path to all vertices from node A are found. (5 marks)
- (b) The formula to compute factorial for n value using **iterative** method and **recursive** method are given below:

$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * (n-1) * (n-2) \dots 3 * 2 * 1 & \text{if } n > 0 \end{cases}$$

Iterative Method to Compute Factorial

$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * \text{factorial}(n-1) & \text{if } n > 0 \end{cases}$$

Recursive Method to Compute Factorial

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVING**Q5. (b) (Continued)**

- (i) Write a function **int iterativeFact(int n)** to compute factorial value for n using **iterative (loop)** method. (4 marks)
- (ii) Write a function **int recursiveFact(int n)** to compute factorial value for n using **recursive** method. (4 marks)

[Total : 25 marks]

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVING**Appendix A: Definition for Node and Linked List**

```
using type = int;
```

```
struct Node {  
    public:  
        type    item;  
        Node    *next;  
        Node(type);  
};
```

```
struct List {  
    int    count;  
    Node    *head;  
    Node    *find(int);  
  
    List();  
    bool empty();  
    int size();  
    bool get(int, type&);  
    bool set(int, type);  
  
    // insert item according to the specified position  
    bool insert(int, type);  
    bool remove(int);  
  
};
```

Appendix B: Definition for Stack Structure

```
using type = char *;
```

```
struct Stack {  
    List list; //Definition is in Appendix A with type = char *  
    bool empty();  
    int size();  
    bool push(type);  
    bool pop(type&);  
  
};
```

UCCD1024 DATA STRUCTURE AND ALGORITHMIC PROBLEM SOLVING**Appendix C: Definition for Queue Structure**

```

using type = char *;

struct Queue {
    int    count;
    Node  *head;
    Node  *tail;
    Queue();
    bool empty();
    int size();
    bool enqueue(type);
    bool dequeue(type&);
    Node  *find(type);
};

```

Appendix D: Partition Function for Quick Sort

```

int partition(int array[], int size) {
    int k;
    int mid = size / 2;
    int index = 0;
    swap(array[0], array[mid]);
    for (k = 1; k < size; k++) {
        if (array[k] < array[0]) {
            index++;
            swap(array[k], array[index]);
        }
    }
    swap(array[0], array[index]);
    return index;
}

```

Appendix E: Definition for BTreeNode, BST, treeFunction and treeFunction2

```

using type = char;

struct BTreeNode {

    type  item;
    BTreeNode  *left, *right;
    BTreeNode(type);
};

struct BST {
    int          count;
    BTreeNode  *root;
};

```

```

    void insert2(BTNode *, BTNode *);
    void case3(BTNode *);
    void case2(BTNode *, BTNode *);
    bool remove2(BTNode *, BTNode *, type);

    BST();
    bool empty();
    int size();
    bool insert (type);           // insert an item into a BST
    bool remove(type);           // remove an item from a BST

};

bool BST::treeFunction()
{
    int h = height(); //height() will return the height of the tree
    bool alt = false;

    if(root == NULL)
        return false;

    for(int i=0; i<=h; i++){
        treeFunction2(root, i, alt);
        alt = !alt;
    }
    return true;
}

void BST::treeFunction2(BTNode *cur, int level, bool alt){

    if(cur == NULL)
        return;

    if(level == 0)
        cout << cur->item << "  ";

    if(level > 0){
        if(alt){
            treeFunction2(cur->left, level - 1, alt);
            treeFunction2(cur->right, level - 1, alt);
        }
        else{
            treeFunction2(cur->right, level - 1, alt);
            treeFunction2(cur->left, level - 1, alt);
        }
    }
}

```